NEPAL COLLEGE OF INFORMATION TECHNOLOGY

Balkumari, Lalitpur

Affiliated to Pokhara University

LAB REPORT FOR EMBEDDED SYSTEMS

notes.arpanadhikari7.com.np

LAB REPORT 2: VHDL CODE (OR & NOT GATE) USING TESTBENCH

Submitted by:

Name: Arpan Adhikari

Roll No.: 231309

BE-CE (5th SEM)

Submitted to:

Er. Simanta Kasaju

Available on notes.arpanadhikari7.com.np

S. N	LAB TOPIC	DATE	REMARKS
1.	VHDL Code (OR & AND Gate)	20th Nov	
2.	VHDL Code (OR & NOT Gate) using testbench	23rd Nov	

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

LAB 2: VHDL CODE (OR & NOT GATE)

OBJECTIVE:

- To develop and simulate a VHDL testbench for verifying the functionality of NOT and OR gate designs.
- To generate all possible input combinations automatically using a testbench.
- To observe and analyze the output waveforms without manually applying inputs.
- To validate that the design under test (DUT) behaves according to its expected truth table.

THEORY:

1. OR Gate

An **OR gate** performs a logical addition operation.

It produces an output **HIGH (1)** when **at least one** of its inputs is HIGH.

The output becomes LOW only when *all* inputs are LOW.

- **Boolean Expression:**
  [
  Y = A + B
  ]
- **Behavior:**
  - 0 OR 0 → 0
  - Otherwise → 1

OR gates are commonly used in circuits where satisfying any one condition triggers the output.

2. NOT Gate

A **NOT gate**, also known as an **inverter**, performs a logical negation operation.

It has a single input and produces an output that is the **inverse** of the input.

- **Boolean Expression:**
  [
  Y = \overline{A}
  ]
- **Behavior:**
  - Input 1 → Output 0
  - Input 0 → Output 1

NOT gates are essential in logic design for generating complements and controlling signal polarity.

Available on notes.arpanadhikari7.com.np

3. VHDL and Digital Gates

In VHDL (VHSIC Hardware Description Language), digital logic gates are described using:

- **Entity:** Defines input and output ports
- **Architecture:** Defines the internal behavior using logical operators (`and`, `or`, `not`)

These constructs allow accurate hardware modeling and simulation before real-world implementation.

4. Testbench in VHDL

A **testbench** is a VHDL code written to test another VHDL design.

It is not synthesized into hardware; instead, it is used only for simulation purposes.

The testbench applies different input combinations automatically and observes the output behavior.

Key Features of a Testbench

- **No entity ports:** Testbenches do not interact with hardware directly.
- **Instantiates the DUT:** The Design Under Test (e.g., AND/OR/NOT gate code) is instantiated inside the testbench.
- **Generates stimuli:** Inputs are applied using signals and time delays.
- **Observes outputs:** Waveforms are viewed to check correctness.
- **Automates testing:** Eliminates the need for manual input during simulation.

Role of Testbench

- Verifies the truth table of logic gates.
- Ensures correct functionality before hardware implementation.
- Helps detect logical errors in the VHDL code.
- Provides complete validation across all possible input combinations.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

VHDL CODE FOR OR GATE:

```
library IEEE;
use IEEE.Std_logic_1164.all;
Entity ArpanOrGate is
  Port(
  A, B : in Std_Logic;
  C : out Std_Logic
  );
End ArpanOrGate;
Architecture behavioral of ArpanOrGate is
begin
  process(A,B)
    begin
    if A='0' or B='0' then
      C<= '0';
  else
    C<='1';
  end if;
  end process;
end behavioral;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

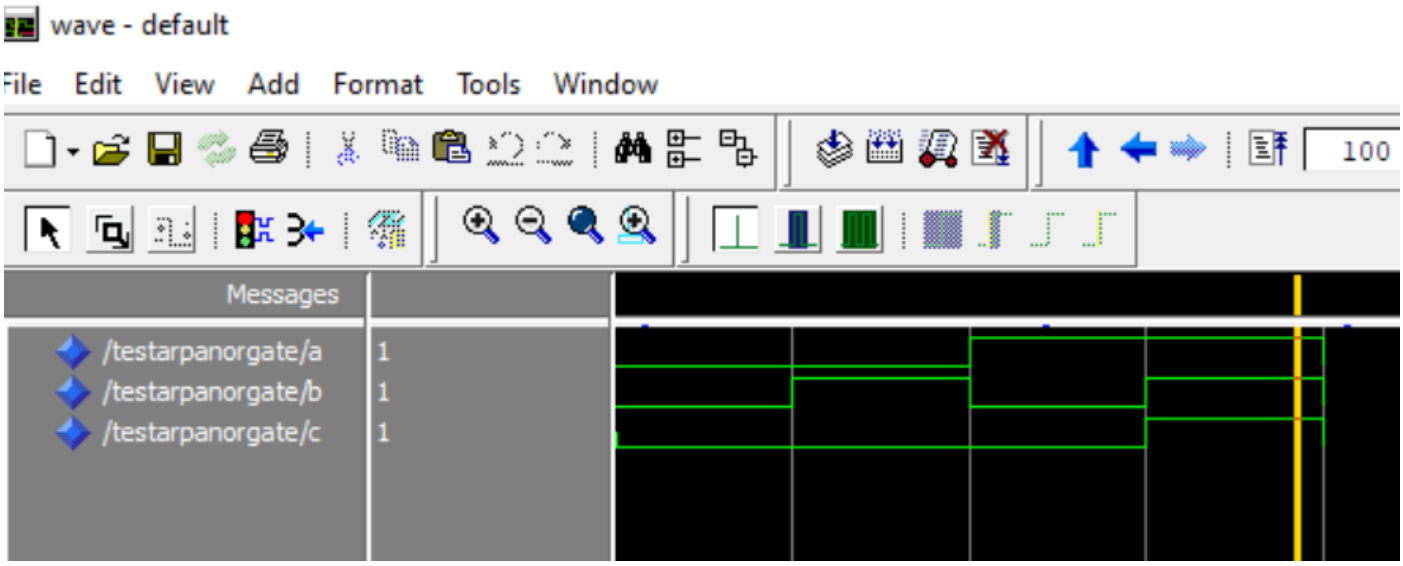
Testbench for OR gate:

```
library IEEE;
use IEEE.std_logic_1164.all;
Entity testArpanOrGate is
End testArpanOrGate;
Architecture behavioral of testArpanOrGate is
Component ArpanOrGate
  Port
  (
  A, B : in Std_logic;
  C: out Std_logic
  );
end component;
Signal A,B : Std_logic := '0';
Signal C : Std_Logic;
begin
UUT: ArpanOrGate port map
(
A => A,
B => B,
C => C
);
stim_proc: process
begin
  A<='0'; B<='0';
  wait for 100 ps;
  A<='0'; B<='1';
  wait for 100 ps;
  A<='1'; B<='0';
  wait for 100 ps;
  A<='1'; B<='1';
  wait for 100 ps;
  end process;
end behavioral;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Waveform for OR gate:



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

VHDL CODE FOR NOT GATE:

```
library IEEE;
use IEEE.std_logic_1164.all;
Entity testArpanNotGate is
End testArpanNotGate;
Architecture behavioral of testArpanNotGate is
Component ArpanNotGate
  Port
  (
  A: in Std_logic;
  B: out Std_logic
  );
end component;
Signal A : Std_logic := '0';
Signal B : Std_Logic;
begin
UUT: ArpanNotGate port map
(
A => A,
B => B
);
stim_proc: process
begin
  A<='0';
  wait for 100 ps;
  A<='1';
  wait for 100 ps;
  end process;
end behavioral;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

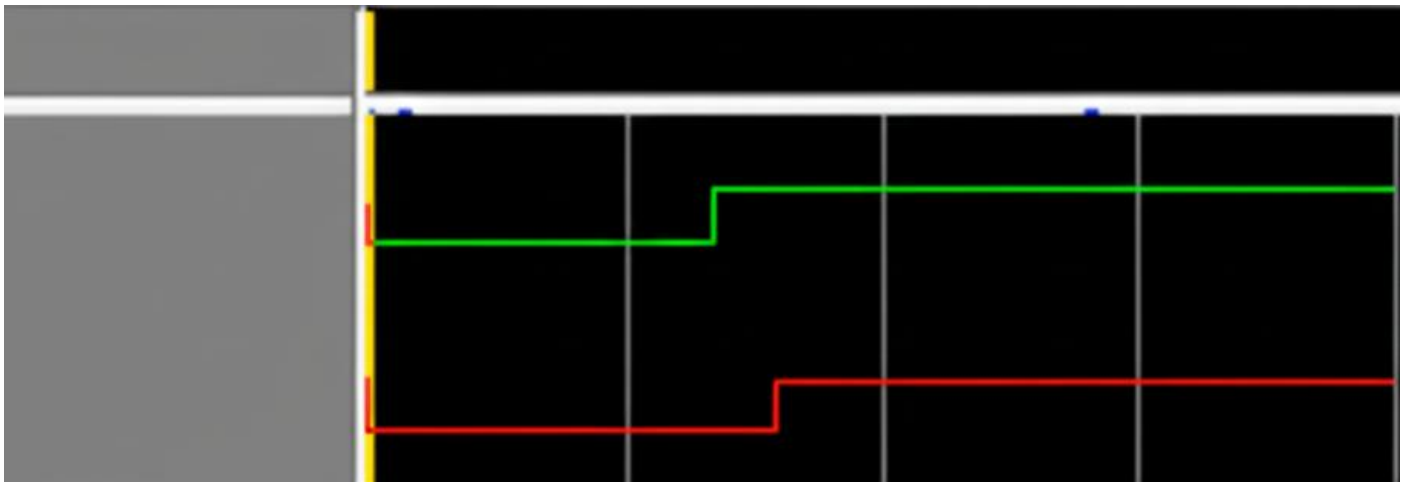
Testbench for NOT gate:

```
library IEEE;
use IEEE.std_logic_1164.all;
Entity testArpanNotGate is
End testArpanNotGate;
Architecture behavioral of testArpanNotGate is
Component ArpanNotGate
  Port
  (
  A: in Std_logic;
  B: out Std_logic
  );
end component;
Signal A : Std_logic := '0';
Signal B : Std_Logic;
begin
UUT: ArpanNotGate port map
(
A => A,
B => B
);
stim_proc: process
begin
  A<='0';
  wait for 100 ps;
  A<='1';
  wait for 100 ps;
  end process;
end behavioral;
```

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

Waveform for NOT gate:



notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np

RESULT:

The VHDL testbench for OR and NOT gates was successfully written and simulated.

The testbench automatically generated all input combinations and applied them to the Design Under Test (DUT).

The obtained simulation waveforms matched the expected truth table:

- The **OR gate** output became HIGH whenever any input was HIGH.
- The **NOT gate** output was always the logical inversion of the input signal.

The waveforms clearly verified that both gates behaved exactly according to their logical definitions.

CONCLUSION:

In this practical, we verified the functionality of OR and NOT gates using a VHDL testbench.

The testbench helped automate the input generation process and provided accurate output waveforms without manual intervention.

The results confirmed that VHDL testbenches are effective tools for validating digital logic circuits, ensuring that the implemented OR and NOT gates function correctly according to their truth tables.

This practical strengthened the understanding of both logic gate behavior and testbench-based simulation in digital design.

notes.arpanadhikari7.com.np

Available on notes.arpanadhikari7.com.np